GROUPE 7

Rapport d'analyse

SÉCURITÉ ANDROID

CODING FACTORY
by ESIEE[tech]
Bachelor Web et Sécurité - Licence 3
35 boulevard du port – 95000 Cergy

US1 - Téléchargement et vérification de l'APK

Objectif principal

Un checksum est une chaîne de caractères calculée depuis la donnée originale. Elle représente le contenu exact de cette donnée.
Si elle change, même s'un seul octet, le checksum généré change également, ce qui va nous permettre de déterminer s'il y a eu corruption ou altération.
C'est pour cette raison qu'il est souvent décrit comme l'empreinte digitale d'un fichier.

Tout d'abord, on va ici **vérifier l'intégrité du fichier app.apk avec un checksum (SHA256).**

```
1  Get-FileHash .\app.apk -Algorithm SHA256

>  F01F08C8B6E2FE4612E81BC7E3A3BA9440DAD0D7A962F4E67640390DD721D528
```

US2 - Analyse automatisée avec MobSF

Objectif principal

MobSF est un outil de sécurité mobile qui permet d'identifier les vulnérabilités et comprendre le fonctionnement interne d'un fichier APK.
En faisant une analyse statique, l'APK est décompilé pour examiner le code, les permissions et les fichiers sensibles. Avec une analyse dynamique, nous avons la possiblité d'observer le comportement de l'application lorsqu'elle est en cours d'exécution (communications réseau/actions suspectes).
À la fin, il génère un rapport détaillé mettant en évidence les failles potentielles, les mauvaises pratiques de sécurité et les éléments critiques pouvant être exploités.

Nous allons ici **lancer une analyse statique afin d'obtenir un résumé des vulnérabilités.**

```
1  docker pull opensecurity/mobile-security-framework-mobsf:latest
2  docker run -it --rm -p 8000:8000 opensecurity/mobile-security-framework-
   mobsf:latest

>  [INFO] Listening at: http://0.0.0.0:8000
```

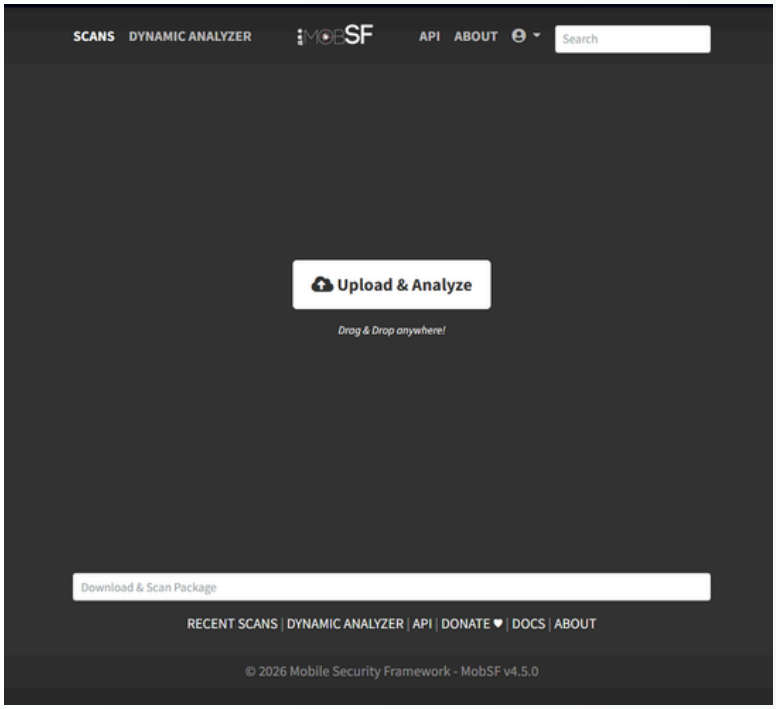
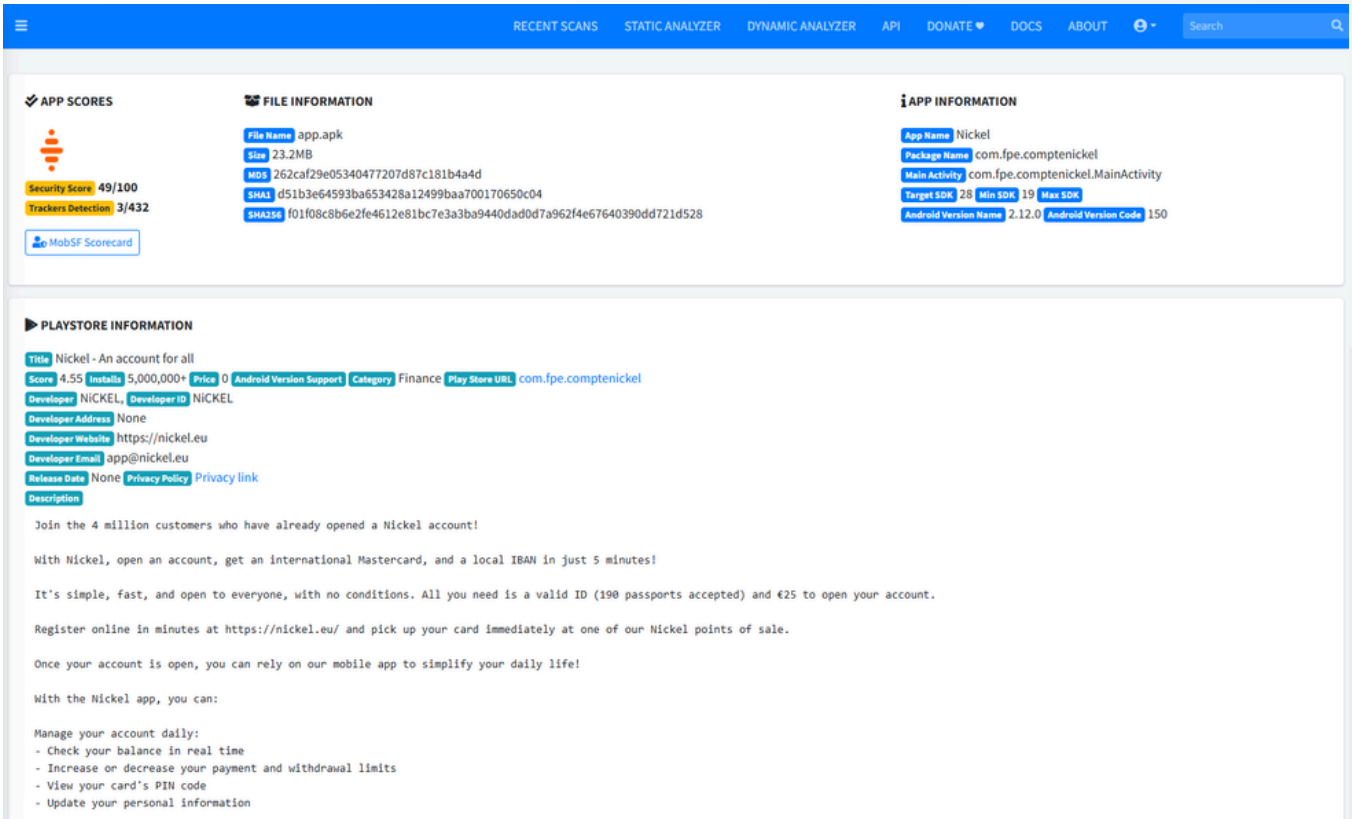


Illustration 1 - Page d'analyse statique MobSF (page d'accueil)

*Dans le cas où la page de login s'affiche, saisir le nom d'utilisateur et mot de passe par défaut suivants :

mobsf
mobsf

Illustration 2 - Dashboard du résumé de l'analyse et des métadonnées



Métadonnées obtenues

On peut voir que le nom de l'application est Nickel, qui est une application bancaire gratuite. La version qui a été analysée est la 2.12.0.

Illustration 3 - Résumé des vulnérabilités

RECENT SCANS | STATIC ANALYZER | DYNAMIC ANALYZER | API | DONATE ♥ | DOCS | ABOUT | Search

0 / 8 EXPORTED ACTIVITIES — View All

3 / 9 EXPORTED SERVICES — View All

5 / 15 EXPORTED RECEIVERS — View All

1 / 12 EXPORTED PROVIDERS — View All

SCAN OPTIONS

Rescan | Manage Suppressions | Start Dynamic Analysis | Scan Logs

DECOMPILED CODE

View AndroidManifest.xml | View Source | View Smali | Download Java Code | Download Smali Code | Download APK

SIGNER CERTIFICATE

```
Binary is signed
v1 signature: True
v2 signature: True
v3 signature: False
v4 signature: False
X.509 Subject: C=fr, ST=idf, L=paris, O=squirrel, OU=dsi, CN=fred
Signature Algorithm: rsassa_pkcs1v15
Valid From: 2016-06-07 23:25:19+00:00
Valid To: 2041-06-01 23:25:19+00:00
Issuer: C=fr, ST=idf, L=paris, O=squirrel, OU=dsi, CN=fred
Serial Number: 0x29be3f8a
Hash Algorithm: sha1
md5: 2688536a0965bd9db93b35eedf35e895
sha1: 4e71ddf7818f86af726767760669ae0da5667c91
sha256: 8a083a1a7c68c28104b60c99e85cda1082d1fdaeb22fa0c5906d25fb73b6ade1
sha512: d0683d82211332ad254c910cac5fd173b822e46d7daca502590c50d3434f3ecb9a217a33c49d1fe9e6faea41810e261ba2defc4a72c60cdda7954e35a10b700f
PublicKey Algorithm: rsa
Bit Size: 2048
Fingerprint: f9d8792a73f54132e135b544666316a08f28ba65256700f47269311950b3c14c
Found 1 unique certificates
```

Illustration 4 - Détail des permissions

RECENT SCANS | STATIC ANALYZER | DYNAMIC ANALYZER | API | DONATE ♥ | DOCS | ABOUT | Search

```
Fingerprint: f9d8792a73f54132e135b544666316a08f28ba65256700f47269311950b3c14c
Found 1 unique certificates
```

APPLICATION PERMISSIONS

Search:

PERMISSION	STATUS	INFO	DESCRIPTION	CODE MAPPINGS
android.permission.READ_CONTACTS	dangerous	read contact data	Allows an application to read all of the contact (address) data stored on your phone. Malicious applications can use this to send your data to other people.	Show Files
android.permission.WRITE_EXTERNAL_STORAGE	dangerous	read/modify/delete external storage contents	Allows an application to write to external storage.	Show Files
android.permission.ACCESS_NETWORK_STATE	normal	view network status	Allows an application to view the status of all networks.	Show Files
android.permission.ACCESS_WIFI_STATE	normal	view Wi-Fi status	Allows an application to view the information about the status of Wi-Fi.	Show Files
android.permission.INTERNET	normal	full Internet access	Allows an application to create network sockets.	Show Files
android.permission.READ_APP_BADGE	normal	show app notification	Allows an application to show app icon badges.	
android.permission.RECEIVE_BOOT_COMPLETED	normal	automatically start at boot	Allows an application to start itself as soon as the system has finished booting. This can make it take longer to start the phone and allow the application to slow down the overall phone by always running.	Show Files
android.permission.USE_FINGERPRINT	normal	allow use of fingerprint	This constant was deprecated in API level 28. Applications should request USE_BIOMETRIC instead.	Show Files
android.permission.VIBRATE	normal	control vibrator	Allows the application to control the vibrator.	
android.permission.WAKE_LOCK	normal	prevent phone from sleeping	Allows an application to prevent the phone from going to sleep.	Show Files

Analyse de sécurité de l'application

L'analyse de l'APK met en évidence un niveau de risque élevé (DANGEROUS) en raison de l'utilisation de permissions sensibles et de plusieurs failles de configuration.

L'application demande notamment l'accès aux contacts via la permission android.permission.READ_CONTACTS, ainsi que l'autorisation d'écriture sur le stockage externe (android.permission.WRITE_EXTERNAL_STORAGE). Cette dernière est particulièrement critique, car elle permet de manipuler des fichiers en dehors du périmètre sécurisé de l'application, sans nécessité fonctionnelle apparente, augmentant ainsi les risques de fuite ou de compromission de données.

Par ailleurs, l'examen du fichier AndroidManifest révèle plusieurs problèmes de sécurité. L'application est compatible avec des versions anciennes d'Android (minSdk = 19, soit Android 4.4), connues pour comporter des vulnérabilités non corrigées. De plus, le trafic en clair est explicitement autorisé (android:usesCleartextTraffic=true), ce qui signifie que des données sensibles telles que mots de passe, adresses e-mail ou autres informations personnelles peuvent être transmises sans chiffrement, les exposant à des interceptions.

L'analyse du code met également en évidence l'utilisation d'un mécanisme de chiffrement reposant sur le mode CBC avec un padding de type PKCS5/PKCS7. Cette configuration est reconnue comme vulnérable à certaines attaques, notamment les padding oracle attacks, pouvant compromettre la confidentialité des données chiffrées.

Enfin, l'intégration de Firebase présente un point de vigilance. Le service Firebase Remote Config est activé et accessible via une API publique, ce qui permet à un tiers d'interroger la configuration distante de l'application. Bien que les données exposées semblent limitées, cela peut révéler des éléments sur la structure interne du projet et doit être encadré afin d'éviter toute exposition d'informations sensibles.

Dans l'ensemble, ces éléments indiquent un niveau de sécurité insuffisant nécessitant des corrections afin de limiter les risques d'exploitation et de protéger les données des utilisateurs.

US3 - Extraction des fichiers

Objectif principal

La décompilation d'un APK consiste à analyser le contenu d'une application Android afin d'en comprendre le fonctionnement interne.

Ici, nous aimerions spécifiquement récupérer AndroidManifest.xml ainsi que classes.dex.

AndroidManifest.xml contient des informations critiques sur l'application (permissions demandées, composants déclarés et configurations de sécurité), ce qui permet d'identifier rapidement des risques potentiels. Le fichier classes.dex regroupe le code exécutable de l'application qui peut être analysé pour comprendre la logique interne, détecter des vulnérabilités ou repérer des comportements malveillants.

Préparation de la décompilation

Voici le protocole que nous avons suivi :

1) Changement de l'extension de APP.APK.
   .apk → .zip
   On obtient donc APP.ZIP.
2) Extraction du dossier compressé
3) Récupération des fichiers AndroidManifest.xml et classes.dex.
4) On récupère le hash des deux fichiers.

```
Get-FileHash .\AndroidManifest.xml, .\classes.dex -Algorithm SHA256

> 80B9B101E3BF02F03EEC4F2C9C9CCF4D61403612E1435949B51CEC8B85D7A276
> 57898C5978715DA96560ED2692B4A8FA1A8E914B37988DCF369CC59951F6C429
```

US4 - Décompilation avec APKTool

Objectif principal

Ici, nous allons extraire l'ensemble des fichiers et dossiers de l'application à partir du fichier .apk d'origine en utilisant Apktool, puis les placer dans un répertoire nommé app_src.

```
apktool d app.apk -o app_src
```

US5 - Analyse de strings.xml

Objectif principal

L'analyse de strings.xml permet d'examiner l'ensemble des chaînes de caractères utilisées par l'application, telles que les textes affichés à l'utilisateur, les messages internes ou certaines valeurs de configuration. Ce fichier a pu nous révéler des informations sensibles comme des clés API, des endpoints et des données de configuration internes. Ces éléments peuvent être exploités par un attaquant pour mieux comprendre l'architecture de l'application, cibler les services backend ou abuser de certaines ressources exposées.

Voici les vulnérabilités qui ont été trouvées dans ce fichier :

```
1  <string name="ACCOUNT_ENDPOINT">https://api.nickel.eu/customer-banking-api</string>
2  <string name="CUSTOMER_AUTHENTICATION_ENDPOINT">https://api.nickel.eu/customer-authentication-
   api</string>
3  <string name="ENVIRONMENT">production</string>
4  <string name="GCM_PROJECT_NUMBER">246766419677</string>
5  <string name="PERSONAL_SPACE_API_ENDPOINT">https://api.nickel.eu/personal-space-api</string>
6  <string name="PUSHWOOSH_APPID">DDF11-D1BF9</string>
7  <string name="build_config_package">com.fpe.comptenickel</string>
8  <string name="com.crashlytics.android.build_id">9ad6d741-4e10-4abd-9f12-163a0e3fb0b2</string>
9  <string name="default_web_client_id">717748501407-
   v621tmfvqd7etdouc3df5vuv31scle0g.apps.googleusercontent.com</string>
10  <string name="firebase_database_url">https://application-client-nickel.firebaseio.com</string>
11  <string name="gcm_defaultSenderId">717748501407</string>
12  <string name="google_api_key">AIzaSyBTgztvImsUfMWDa41PCrDWAj7dmyIDhUg</string>
13  <string name="google_app_id">1:717748501407:android:e0cdeb712e27275d</string>
14  <string name="google_crash_reporting_api_key">AIzaSyBTgztvImsUfMWDa41PCrDWAj7dmyIDhUg</string>
15  <string name="google_storage_bucket">application-client-nickel.appspot.com</string>
```

Vulnérabilités

- Exposition d'endpoints API en clair, permettant une interaction directe avec les services backend.
- Présence de clés API et d'identifiants (Google, Firebase) accessibles côté client, pouvant être abusés.
- Divulgation de la configuration Firebase (URL, stockage), facilitant l'accès ou l'énumération des données.
- Indication de l'environnement de production, aidant un attaquant à cibler des données réelles.
- Exposition d'identifiants de services tiers (Pushwoosh), pouvant être utilisés de manière malveillante.
- Présence d'informations internes (package, build), facilitant la rétro-ingénierie.
- Utilisation d'un chiffrement vulnérable (CBC avec PKCS5/PKCS7), exposé aux attaques de type padding oracle.
- Configuration permissive (trafic en clair autorisé), exposant les données sensibles à l'interception.

US6 - Analyse de code

Objectif principal

L'analyse du code après décompilation consiste à examiner le code source reconstruit afin de comprendre la logique interne de l'application et d'identifier d'éventuelles failles de sécurité. Cette étape permet notamment de repérer des pratiques dangereuses telles que l'utilisation de méthodes de chiffrement faibles, la présence de données sensibles en dur, des contrôles d'accès insuffisants ou encore des appels à des fonctions critiques mal sécurisés. Elle offre également une meilleure compréhension des interactions avec les API, des mécanismes d'authentification et du comportement global de l'application.

Nos findings

Voici nos findings après avoir conduit l'analyse de code :

1) Certificate pinning affaibli :

Le composant PinningTrustManager utilise SHA-1 pour vérifier les pins, ce qui est une mauvaise pratique de sécurité.
En plus, si les pins sont trop anciens, le code revient à la confiance système, ce qui réduit la protection du pinning.

2) Credentials renvoyés en clair :

Dans KeychainModule, le username et le password sont déchiffrés puis renvoyés en clair dans une WritableMap.
Cela augmente l'exposition des secrets en mémoire et entre la couche native Android et la couche React Native.

3) Stockage local dans SharedPreferences :

Dans PrefsStorage, les identifiants sont stockés dans SharedPreferences avec un encodage Base64.
Base64 n'est pas un chiffrement, donc la sécurité dépend entièrement de la couche de chiffrement utilisée avant le stockage.

4) Persistance des cookies :

ForwardingCookieHandler lit, écrit et sauvegarde les cookies avec CookieManager.
Si ces cookies contiennent des informations de session, leur conservation locale augmente la surface d'exposition côté client.

US7 - Recompilation de l'APK

Objectif principal

La recompilation consiste à reconstruire l'application à partir du code précédemment décompilé en utilisant Apktool, afin de tester, modifier ou analyser son fonctionnement.

Voici la commande utilisée :

```
apktool b app_src -o app_rebuilt.apk
```

US8 - Signature de l'APK

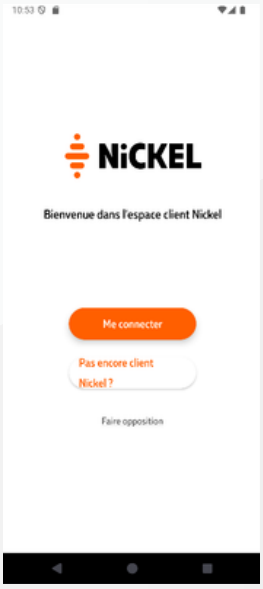
Objectif principal

Pour pouvoir installer une APK recompilée sur un émulateur ou un appareil Android, il est nécessaire de la signer avec une clé valide, car Android refuse les APK non signées pour des raisons de sécurité.

Nous pouvons alors installer l'application sur un émulateur ou un appareil Android pour effectuer des tests. Cela garantit que le système Android acceptera l'APK et que toutes les modifications apportées lors de la recompilation ne compromettent pas la validité de la signature.

Voici la preuve de la signature ainsi que l'application lancée sur un émulateur :

```
PS C:\Users\fesco\esiee-it\cyber\Cybers-cu\story7> & "C:\Users\fesco\AppData\Local\Android\Sdk\build-tools\36.1.0\apksigner.bat" verify app_recompiled.apk
WARNING: META-INF/android.arch.work_work-runtime.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or move
 the entry outside of META-INF/.
WARNING: META-INF/androidx.activity_activity.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or move the
 entry outside of META-INF/.
WARNING: META-INF/androidx.appcompat_appcompat-resources.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete
 or move the entry outside of META-INF/.
WARNING: META-INF/androidx.appcompat_appcompat.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or move t
he entry outside of META-INF/.
WARNING: META-INF/androidx.arch.core_core-runtime.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or mov
e the entry outside of META-INF/.
WARNING: META-INF/androidx.asynclayoutinflater_asynclayoutinflater.version not protected by signature. Unauthorized modifications to this JAR entry will not be detect
ed. Delete or move the entry outside of META-INF/.
WARNING: META-INF/androidx.cardview_cardview.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or move the
 entry outside of META-INF/.
WARNING: META-INF/androidx.coordinatorlayout_coordinatorlayout.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected.
Delete or move the entry outside of META-INF/.
WARNING: META-INF/androidx.core_core.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or move the entry o
utside of META-INF/.
WARNING: META-INF/androidx.cursoradapter_cursoradapter.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete o
r move the entry outside of META-INF/.
WARNING: META-INF/androidx.customview_customview.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or move
 the entry outside of META-INF/.
```



US9 - Vérification Firebase

Objectif principal

Nous aimerions vérifier si la base de données en ligne est accessible ou modifiable par des utilisateurs non autorisés, en exploitant les URL de configuration présentes dans le projet.

Nos findings

Lors de l'analyse des endpoints Firebase, nous avons testé l'URL suivante :

```
https://firebaseremoteconfig.googleapis.com/v1/projects/717748501407/namespaces/firebase:fetch?key=AIzaSyBTgztvImsUfMWDa41PCrDWAj7dmyIDhUg
```

Cette requête renvoie une erreur 404 Not Found, ce qui indique que l'endpoint n'est pas accessible publiquement. Il n'a donc pas été possible de récupérer les paramètres de configuration du projet Firebase, probablement parce que le projet a été supprimé après une longue période d'inactivité.

Nous avons également testé l'URL du console Firebase :

```
https://console.firebase.google.com/u/0/project/application-client-nickel/database/application-client-nickel/data/?pli=1
```

Dans ce cas, le navigateur affiche un message indiquant que le projet n'existe pas ou que l'accès n'est pas autorisé. Les tests avec curl ne renvoient aucune information. Là encore, la suppression du projet par Google pour cause d'inactivité explique l'absence d'accès.

En conclusion, si le projet Firebase avait été actif et accessible, il aurait été possible d'examiner les données de la base et d'identifier les clés API présentes dans les fichiers du projet. Cependant, comme le projet n'est plus disponible, aucune donnée sensible n'est exposée et il n'y a pas de problème de sécurité lié à ces endpoints.

US11 -Politique de sécurité mobile

Notre politque

Nous avons mis au point une politique globale pour renforcer la sécurité des applications mobiles et leur utilisation au sein d'une entreprise :

L'entreprise fournit des téléphones professionnels et interdit l'utilisation de téléphones personnels afin de garantir la confidentialité des données sensibles. L'installation d'applications depuis les boutiques publiques est formellement interdite ; seules les applications nécessaires au travail peuvent être téléchargées via le site officiel de l'entreprise.

1) Sécurité et gestion des appareils :

Chaque téléphone professionnel doit être protégé par un code PIN fourni par l'entreprise, qui ne doit jamais être désactivé. Les appareils sont gérés via un système MDM (Mobile Device Management) permettant un contrôle centralisé et une sécurité renforcée.

2) Mise à jour des appareils :

Il est obligatoire de maintenir tous les téléphones et PC professionnels à jour avec les dernières versions logicielles et correctifs de sécurité afin de prévenir toute vulnérabilité exploitée par des attaquants.

3) Sauvegarde des projets :

Tous les projets doivent être régulièrement sauvegardés sur les systèmes de stockage sécurisés de l'entreprise pour éviter toute perte de données et garantir la continuité des activités.

4) Politique de développement sécurisé :

- Protection des clés et secrets : ne jamais stocker les clés API, identifiants Firebase ou autres secrets en clair dans le code ou les fichiers de configuration (ex : strings.xml), et utiliser des mécanismes sécurisés côté serveur.
- Stockage sécurisé : les données sensibles ne doivent pas être stockées en Base64 ou en clair dans SharedPreferences ; un chiffrement robuste et validé doit être utilisé.
- Certificate pinning : utiliser des algorithmes modernes (SHA-256 ou supérieurs) et maintenir les pins à jour pour éviter le retour à la confiance système.
- Transmission des données : toutes les communications réseau doivent être chiffrées et le trafic en clair est strictement interdit.
- Gestion des cookies et sessions : éviter la persistance locale de cookies contenant des informations sensibles et s'assurer que leur stockage est sécurisé.
- Séparation des couches : les identifiants et mots de passe ne doivent jamais être transmis en clair entre la couche native et React Native ou toute autre couche applicative.
- Revue de code et audits réguliers : mettre en place des revues de code systématiques et des audits de sécurité réguliers pour détecter et corriger les vulnérabilités, telles que celles identifiées lors des analyses (padding oracle, endpoints exposés, API keys visibles).

US12 - Choix d’une solution MDM

Objectif principal

Nous avons également comparé deux solutions MDM afin d’identifier et de recommander celle qui offre la meilleure sécurité et gestion la plus efficace des appareils mobiles ainsi que la conformité aux besoins de l’entreprise.

Comparaison entre Microsoft Intune et Knox Manage

Microsoft Intune et NinjaOne sont deux solutions MDM permettant de gérer et sécuriser les appareils mobiles professionnels.

Intune, intégré à l'écosystème Microsoft 365 et Azure AD, offre une gestion centralisée des appareils, des politiques de conformité et une forte intégration avec les services Microsoft, mais son déploiement peut être complexe et coûteux pour les petites organisations.

NinjaOne propose une plateforme unifiée et facile à prendre en main, avec un déploiement rapide, une gestion centralisée des applications et des fonctionnalités de sécurité robustes, bien que certaines fonctions avancées puissent nécessiter une formation pour être pleinement exploitées.

Le choix entre ces solutions dépendra de l'infrastructure existante, des besoins de sécurité et des ressources disponibles pour la gestion des appareils :

Catégories	Microsoft Intune	NinjaOne (MDM)
Architecture	Cloud Microsoft natif	SaaS cloud unifié
Gestion des identités	Intégration native et profonde avec Azure AD	Intégration via connecteurs, orientée gestion flotte
Déploiement	Autopilot (Windows), Apple Business Manager, Android Enterprise	Via portail, QR code ou intégration Apple/Google
Sécurité	Accès conditionnel, chiffrement, suppression à distance	Verrouillage, effacement à distance, réinitialisation de code rapide
Gestion applicative	Déploiement granulaire (MAM) : Word, Excel, etc.	Distribution d'apps et gestion des politiques en masse
Facilité d'usage	Complexe : nécessite des compétences	Facile : prise en main rapide
Points forts	Intégration totale avec Microsoft 365	Déploiement rapide
Inconvénients	Interface lourde pour de petites tâches	Contrôle limité sur certaines applications